1. Cmdlets, Parameter und PowerShell-Hilfe

https://learn.microsoft.com/de-de/powershell/scripting/learn/ps101/01-getting-started?view=powershell-7.4

`PS <path> $PSVersionTable` ... liefert z.B. `PSVersion 7.5.1`

Unterschied Windows PowerShell (Version 5.x wird mit `powershell` gestartet) und Microsoft PowerShell (Version 7.x wird mit `pwsh` gestartet)

Im Windows Terminal kann pro Tab ausgewählt werden, welche PowerShell Version verwendet werden soll.

Installation mit `winget search Microsoft.PowerShell` (liefert zuerst eine Liste der neuesten Versionen) und dann installieren mit der Id `winget install --id Microsoft.Powershell --source winget`

Übung 1.1: Cmdlets finden

Powershell ist der Ersatz für die klassische Windows-Kommandozeile, die `cmd.exe`. Prinzipiell kann Powershell fast alles, was die Kommandozeile auch kann, wie zum Beispiel die IP-Adresse mit dem Befehl `ipconfig.exe` anzeigen.

Um die besonderen Fähigkeiten der Powershell nutzen zu können, müssen aber die Powershell-eigenen Befehle, genannt **Cmdlets** (Commandlets gesprochen), verwendet werden. Cmdlets sind **kleine Programme** (eben "Kommandochen"), die jeweils eine **spezialisierte Aufgabe** erledigen.

Ein Cmdlet besteht aus einem **Verb**, das durch einen **Bindestrich** mit dem **Ding** (= Subjekt) verbunden ist, mit dem es etwas tun soll. Sie haben im Gegensatz zu klassischen Kommandos, die meist "eierlegende Wollmilchsäue" (= Schweizertaschenmesser/Swiss army knife) sind, zwei Vorteile: Sie sind durch ihre **eingeschränkte Funktion einfacher zu bedienen** und durch ihre **Namensgebung einfach zu finden**.

Das Finden erledigt dabei das Cmdlet `Get-Command`, das hilft, die zur Verfügung stehenden Kommandos zu suchen und zu filtern. Hier ein paar Beispiele für Cmdlets:

- `Get-Verb`: zeigt alle Verben an, aus denen ein Cmdlet-Name bestehen kann
- `Get-Alias`: zeigt alle Aliasse (Kurznamen) an, die auf ein Cmdlet verweisen
  z.B.: `Get-Alias -Name type` liefert das Cmdlet `Get-Content`
- Update-Help`: aktualisiert die Powershell-Hilfe
- `Out-File`: leitet die Ausgabe von Powershell in eine Textdatei aus

Mit `Get-Command` bekommt man alle Kommandos ausgegeben, die die Powershell des Rechners zur Verfügung stellt. Zum Suchen kann ein Filter verwendet werden. Der Filter versteht auch **Wildcards** (= Platzhalter), also das `*` für eine beliebige Anzahl beliebiger Zeichen, sowie das `?` als Platzhalter für genau ein beliebiges Zeichen.

Der Filter muss hinter dem Parameter `-Name` angegeben werden, zum Beispiel so:

```
Get-Command -Name Get*
```

Dabei bitte darauf achten, dass `-Name` mit einem **Leerzeichen** von `Get-Command` getrennt sein muss, genauso wie der Filterbegriff. Das obige Beispiel gibt alle Kommandos aus, die mit dem Verb `Get` anfangen.

Ein sehr praktischer kleiner Trick: Mit der Tabulator-Taste kann man sich die Kommandonamen und Parameter auflösen lassen. Bei Eingabe von `Get-Co` und einmal `Tab` drücken, ergänzt Powershell den Rest des Befehls.

Ein weiteres Auslösen von Tab zeigt den nächsten Befehl an, der mit `Get-Co` anfängt und so weiter, bis man wieder bei `Get-Command` angekommen ist. Und noch ein wichtiger Tipp: Obwohl der Tabulator alle Verben und Dinge immer mit einem führenden Großbuchstaben auflöst, unterscheidet Powershell **nicht zwischen Groß- und Kleinschreibung**.

PowerShell-Sitzung protokollieren

Um eine PowerShell-Sitzung zu protokollieren (= Transkript erstellen), verwenden wir das Cmdlet `Start-Transcript` zum Starten und `Stop-Transcript` zum Beenden der Aufzeichnung.
Das Transkript speichert alle eingegebenen Befehle und die Konsolenausgabe in einer Textdatei, was für Dokumentation, Fehlerbehebung oder zur Weitergabe von Informationen nützlich ist

Starte das Transkript mit folgendem Cmdlet und setze dabei deinen `<Username>` (z.B. `RAH`) ein. Damit wird dein Transkript im aktuellen Verzeichnis erstellt.

```
Start-Transcript -FilePath "SYT2BS_<Username>_$(Get-Date -Format yyyyMMdd-HHmmss).log"
```

Am Ende der Stunde bzw. am Ende deiner Aufzeichnungs-Session wird das Transkript mit folgendem Cmdlet beendet:

```
Stop-Transcript
```

Den Inhalt deines Transkripts kannst du dir dann mit dem folgenden Cmdlet ausgeben:

```
Get-Content "SYT2BS_<Username>_<Datum der Aufzeichnung>.log"
```

Und jetzt die Übung: Führe die Aufgaben mit dem Cmdlet `Get-Command` durch. Suche zuerst damit das richtige Cmdlet mit dem du dann die jeweilige Aufgabe lösen kannst.

- **Ü1.1.1** Finde zu allen bereits bekannten Befehlen aus der klassichen Windows-Kommandozeile (= cmd) die "neuen" PowerShell Cmdlet. Benutze dazu das Cmdlet `Get-Command` und verwende den entsprechenden Parameter, welcher dir die Ausgabe auf "Aliasse" filtert. Notiere dir jetzt die bekannten Kommandozeilenbefehle und das entsprechende Cmdlet in der folgenden Tabelle.
  Deine Lösung:

  ```
  Get-Command -CommandType Alias
  ```

Nr.	Befehl Windows-Kommandozeile (alias)	Powershell Cmdlet
1	copy	Copy-Item
2	cd	Set-Loaction
3	echo	Write-Output
4	history	Get-History
5	type	Get-Content
6	pwd	Get-Location
7	rd	Remove-Item
8	set	Set-Variable
9	move	Move-Item
10	cls	Clear-Host

- **Ü1.1.2** alle Prozesse (= laufende Programme) auflisten
  Deine Lösung:

```
Get-Process
```

- **Ü1.1.3** alle lokalen Benutzer auflisten
  Deine Lösung:

```
Get-LocalUser
```

- **Ü1.1.4** alle Dateien im Ordner `...\SJ202526` auflisten (Hinweis: Dateien und Ordner werden als **Items** bezeichnet)
  Deine Lösung:

```
Get-ChildItem -File
```

- **Ü1.1.5** den aktuellen Inhalt der Zwischenablage anzeigen (Hinweis: die Zwischenablage wird als Clipboard bezeichnet) und mit dem Inhalt `Mein Name lautet <Vorname> <Nachname>.` überschreiben.
  Deine Lösung:

```
Get-Clipboard && Set-Clipboard "Mein Name lautet Paul Sommer."
```

- **Ü1.1.6** Gib das aktuelle Datum und die aktuelle Uhrzeit aus.
  Deine Lösung:

```
Get-Date
```

Übung 1.2: Parameter verwenden

Die Cmdlets, die wir bisher kennengelernt haben, können oft noch viel mehr als ihre reine Basisfunktionalität. Um die vollen Fähigkeiten eines Cmdlets ausreizen zu können, gibt es Parameter. Parameter sind Befehle, die einem Cmdlet mitgegeben werden.

Mit ihnen kann man zum Beispiel die Ausgabe eines Cmdlets einschränken, indem man dem Cmdlet mitteilt, welche Daten man genau bekommen möchte.

Ein Parameter besteht meist aus dem Parameter-Namen, der festlegt, welche Funktion du konfigurieren möchtest, und dem Argument oder Wert, welcher festlegt, was eingestellt werden soll. Hier ein Beispiel:

```
Get-ChildItem -Path c:\Windows -Name *.exe
```

`Get-ChildItem` zeigt alle Dateien eines Ordners an. Wird kein Pfad (Path) angegeben, wird der Inhalt des Ordners angezeigt, in dem sich die Powershell gerade befindet. Der aktuelle Ort steht normalerweise am Anfang jeder Zeile, angeführt von einem PS.

Im obigen Beispiel gibt es zwei Parameter: `-Path` und `-Name`. Der Parameter `-Path` gibt den Pfad an, dessen Inhalt man sehen möchte. `-Name` gibt den Namen der Datei an, die angezeigt werden soll. Beide Parameter verfügen über ein Argument (den Wert, der an den Parameter übergeben wird).

Das Argument des Pfades ist `c:\Windows` – es sollen also alle Dateien aus dem Windows-Ordner angegeben werden. Das Argument `-Name` gibt den Namen der Dateien an, die angezeigt werden sollen. `*` ist hier ein Platzhalter.

Er bedeutet: Zeige alle Dateien an, die mit beliebigen Zeichen anfangen, aber mit dem Text `.exe` enden.
Hinweis: Ein Trick, um sich in Powershell alle verfügbaren Parameter anzeigen zu lassen, ist, ein Leerzeichen gefolgt von einem – (Bindestrich) hinter dem Cmdlet-Namen einzugeben und dann *Strg* (Steuerung) + *Leerzeichen* (Space) einzugeben. Powershell zeigt dann alle Parameter an, die es gibt.

Arten von Parametern:

- **Singlevalue-Parameter:** `Get-Date -Format "dddd MM/dd/yyyy HH:mm K"` ... ein Parameter-Name mit genau einem Argument
- **Multivalue-Parameter:** `Get-Command -Verb get, set` ... ein Parameter-Name mit mehreren Argumenten (duch Komma getrennt).
- **Switch -Parameter:** `Get-Command -All` ... ein Parameter-Name ohne Argument, schaltet ein oder aus, wirkt wie ein Schalter

Die Aufgaben:

- **Ü1.2.1** Finde alle Dateien und Ordner im aktuellen Verzeichnis, die mit S anfangen und auf `.log` enden.

  Deine Lösung:

  ```
  Get-ChildItem S*.log
  ```

- **Ü1.2.2** Finde alle Verben, die mit A anfangen.
  Deine Lösung:

```
Get-Verb A*
```

- **Ü1.2.3** Finde alle lokalen Benutzer, die mit a anfangen (normalerweise der *administrator*).
  Deine Lösung:

```
Get-LocalUser A*
```

- **Ü1.2.4** Gib das Cmdlet `Get-Date -Month 4 -Day 31` ein und beschreibe was passiert:
  Deine Lösung:

```
Thursday, 1. May 2025 12:47:50
Es wird der Tag und Monat vom aktuellen Datum überschrieben mit den arguments vom Command
Und da April keine 31 Tage hat, spring es auf den 1 Mai
```

Übung 1.3: Die Powershell-Hilfe benutzen

Manchmal ist es schwierig zu verstehen, wie ein Cmdlet funktioniert. Zum Glück hat Powershell eine eingebaute Hilfe. Die Hilfe wird mit dem Cmdlet `Get-Help` aufgerufen. Mit dem Parameter `-Name <Cmdlet-Name>` kann man sich die Hilfe zu einem Cmdlet anzeigen lassen.

```
Get-Help -Name <Cmdlet>
```

Hinweis: Vor der ersten Verwendung muss die Hilfe einmal aus dem Internet aktualisiert werden. Dafür die Powershell mit Administrativen Rechten starten und `Update-Help` aufrufen. Alternativ kann man auch direkt `Get-Help` starten – Powershell weist bei der ersten Anmeldung dann darauf hin, dass die Hilfe noch heruntergeladen werden muss, und bietet direkt an, die Hilfe zu aktualisieren.

Das Cmdlet `Get-Help` zeigt normalerweise nur eine reduzierte Hilfe an. Die vollständige Hilfe bekommt man mit dem Parameter `-ShowWindow`. Die Hilfe wird dann in einem Fenster geöffnet.

```
Get-Help -Name Get-Command -ShowWindow
```

Der Parameter `-ShowWindow` ist übrigens ein Switch-Parameter oder Schalter. Er heißt so, weil man kein Argument übergeben muss (und auch gar nicht kann) – er ist einfach nur an oder aus.

Die Hilfe zeigt an, welche Parameter es gibt und wie sie eingesetzt werden können. Außerdem wird die gesamte Funktion des Cmdlets beschrieben. Im Abschnitt Beispiele stehen außerdem praktische Anwendungsfälle mit Beschreibung, die zeigen, wie das Cmdlet eingesetzt werden kann – das ist meist sehr, sehr hilfreich!

Alternativ zu `-ShowWindows` kann man auch `-Online` verwenden. Powershell öffnet dann einen Webbrowser und zeigt die Hilfe dann auf einer Webseite an.

```
Get-Help -Name Get-Command -Online
```

Die Aufgabe:

Mit `Get-Command` sollen die nachstehenden Aufgaben gelöst werden; dazu werden Parameter benötigt, welche du mit der PowerShell-Hile nachschauen kannst.

- **Ü1.3.1** Lass dir alle laufenden Prozesse (= ausgeführte Programme) in der Konsole anzeigen.
  Deine Lösung:

```
Get-Process
```

- **Ü1.3.2** Starte Microsoft Excel auf deinem Laptop. Suche zuerst nach allen Prozessen, wo im Namen "Excel" vorkommt. Suche nach dem gleichen Prozess und verwende jetzt die Process-ID (= PID).
  Deine Lösung:

  ```
  Get-Process -Name *Excel*
  ```

- **Ü1.3.3** Beende das geöffnete Microsoft Excel wieder.
  Deine Lösung:

  ```
  Stop-Process -Name *Excel*
  ```

- **Ü1.3.4** Erstelle einen neuen Ordner unter SJ202526/SYT2-BS mit dem Namen `PwshScripts`.
  Ein kleiner Tipp: Zum Abfragen von Elementen aus einem Ordner verwendet man das Cmdlet `Get-ChildItem`. Das Kommando zum Erstellen von einzelnen Elementen hat einen ähnlichen Namen.
  Deine Lösung:

  ```
  New-Item ~/OneDrive` -` HTL` Hollabrunn/SJ202526/SYT/BS/PwshScripts -ItemType "Directory"
  ```